

ESOF 322

Software Engineering

LECTURE: 1

J. L. Pach

OUTLINE:

Syllabus,

Textbook

Canvas

Introduction

SYLLABUS

SOME BASIC FACTS ABOUT THE COURSE

Course Name:

Software Engineering (ESOF 322)

Credit Hours:

3 credits

1 hour lecture triple a week

Lecture (Mondays, Wednesdays, Fridays)

10:00 – 10:50 AM

ENGR LAB CLASSROOM BLDG. 315 (S&E) 315

SYLLABUS

Course Description

Textbooks

Class Rules

Grading

Accommodations & Academic Dishonesty

Declaration of authorship

COURSE DESCRIPTION

This course introduces the processes and practices of software engineering. Topics include software process models, metrics, requirements engineering, design, testing, quality assurance, configuration management, and software inspections. Additional emphasis is placed on code documentation, maintainability, and individual problem-solving. Students will work on practical case studies and focused programming assignments. Selected aspects of reverse engineering and software security are also introduced to broaden the perspective on modern software development challenges.

…A FEW WORDS

I defended my Master's thesis in Computer Science in 2012. At that time, many of the key concepts in software engineering were already well established. I studied UML, design patterns, and used the very same textbook that we will be referencing in this course. However, since then, the field has changed dramatically. Tools like GIT and GitHub, JSON as a standard communication format, Visual Studio Code, makefiles, and many others have not only evolved but have become industry standards rather than curiosities.

When I took this course, it was taught in a very theoretical way. My lecturer had never worked in a software company, and Sommerville's textbook—while a gold standard in the field—often felt too abstract and impractical. I personally remember the course as more of an obligation than an inspiration, with an overwhelming focus on UML without showing its real-world applications.

…A FEW WORDS2

That is why I want to approach this course differently. My goal is to make theory and practice complement each other. Sommerville discusses many important issues, but often at the level of a software engineer as a career role—positions that are usually reached after many years of experience, climbing from junior to senior developer and eventually to architect or engineer.

What you need right now are practical skills: working with tools, understanding and improving existing code, and adding functionality based on documentation rather than creating it from scratch. For this reason, we will put more emphasis on problem-solving than on dry theory.

My aim is to combine theory with practice—similar to how computer architecture is best understood: knowledge of registers and assembly language only makes sense when you truly understand how the x86 processor works.

TEXTBOOKS

Sommerville, I. (2016). Software engineering 10th Edition. ISBN-10, 1292096136 (Main)

ROAD TRIP

Programmer's Toolbox

Software Development Process

Architecture and Patterns

Advanced Topics

PROGRAMMER'S TOOLBOX

Git (repository creation, commit, branching, merging, refactoring, pull requests)

GitHub (upload, clone, README.md – introduction to Markdown)

Markdown + Mermaid (first Markdown basics, then simple diagrams – flowchart)

Virtual environments in Python (venv, .env)

Compilers, makefile, JSON, configuration in VS Code

Debugging (basics in Python + IDE tools)

SOFTWARE DEVELOPMENT PROCESS

Plan-driven methods (Waterfall, V-model)

Agile methods (Scrum, Kanban)

Extreme Programming (pair programming, refactoring, TDD)

Unit testing (pytest in Python)

ARCHITECTURE AND PATTERNS

Architectural models (layered, client-server, repository, microservices – overview)

Software patterns (design vs. architectural – Singleton, Strategy, MVC)

OOP + UML class diagrams (OOP paradigms and class documentation)

UML diagrams (use case, sequence, activity)

UML in Mermaid (reproducing these diagrams in Markdown)

ADVANCED TOPICS

Reverse engineering, security, certification (Cheat Engine as a case study)

REST API (simple GET/POST)

THE ORDER OF THE TOPICS

The order of the topics in this course is not rigid. We will continuously blend theory with practical applications to keep the material as interesting and engaging as possible.

Our goal is to ensure you can immediately apply the knowledge you gain.

GRADING BREAKDOWN

** The final grade will be calculated based on the following components:

Quizzes (25%):

Entrance Quizzes: Short quizzes (5-10 minutes) may be given at the beginning of some classes to assess understanding of the previous material.

Two Major Quizzes: 50-minute quizzes administered throughout the semester. Each student will have one opportunity to retake each quiz at the end of the semester.

Assignments (75%): Regular, individual assignments will primarily be completed during class time;
The final grade will be calculated as a weighted average of the scores obtained in each component.**

IN-PERSON FORMAT

This is strictly an in-person course. Regular attendance and active participation are expected.

Attendance Policy

Attendance at every class is required.

Students are allowed up to two unexcused absences during the semester without penalty.

Each additional unexcused absence beyond this two-absence allowance will result in a 1 percentage-point deduction from the final course grade.

EXCUSED ABSENCES & EMERGENCIES

Official university-excused absences, university-sanctioned activities, serious personal emergencies, and documented medical circumstances will not count toward the unexcused absence allowance.

Students should notify the instructor as soon as reasonably possible when an emergency or officially excused absence occurs.

LABORATORY RULES

Entrance Quiz

When an entrance quiz is scheduled, it will consist of three questions.

A score of at least 2 out of 3 is required to pass.

A failed entrance quiz may be retaken up to two times during the semester. Only failed entrance quizzes may be retaken.

ASSIGNMENTS & DEADLINE

Students will have six calendar days to complete and submit each assignment.

The deadline is strict. Once the six-day submission period has closed, the normal submission path will be closed and late or makeup submissions will not be accepted, except where an official University policy, documented emergency, or approved accommodation requires otherwise.

Students are responsible for submitting their work before the deadline. Students should not wait until the final minutes before the deadline to submit an assignment.

Brief Concluding Quiz

When scheduled, the concluding quiz is a short summative assessment covering the material addressed during the laboratory session.

CODE FORMATTING, AUTHORSHIP & DECLARATION

Every submitted source file must begin with exactly four lines of comments containing the required authorship declaration.

The following template must be used:

```
// Your Name
// ESOF 322 Fall 2026
// Programming Assignment #1
// I declare that I am the author of this work, take full responsibility for it, and have disclosed any material external assistance.
```



AUTHORSHIP REQUIREMENT

The four-line declaration is **mandatory**.

A source file submitted without the required declaration will receive **0 points**.

If a student submits an assignment before the deadline but accidentally omits the required declaration, the instructor may allow the student to resubmit **the same code with the declaration added** after the deadline as a correction of an administrative omission.

This correction is limited strictly to adding the required declaration. No modification, improvement, debugging, or other change to the submitted code is permitted after the original deadline.

Repeated failure to include the required declaration may be treated as failure to comply with the assignment requirements.

ACADEMIC INTEGRITY, COLLABORATION & EXTERNAL RESOURCES

Students are encouraged to use appropriate external resources to learn and solve problems. Such resources may include:

- textbooks and other books;
- official programming documentation;
- technical websites and documentation;
- Stack Overflow and similar technical resources;
- ChatGPT, Gemini, GitHub Copilot, and other AI-assisted tools.

The use of an external resource does not, by itself, constitute academic misconduct. However, students remain fully responsible for the work they submit.

DISCLOSURE OF EXTERNAL ASSISTANCE

Students must disclose **material external assistance** that contributed to their submitted work.

Such assistance may include, but is not limited to, substantial assistance from another person, technical resources, or generative AI tools.

The disclosure should be made in an appropriate comment in the source code.

For example:

```
// I used the C standard library documentation to verify the behavior of strtok().
```

or:

```
// I used ChatGPT to help explain pointer arithmetic.
// I wrote, tested, and verified the submitted implementation myself.
```

DISCLOSURE OF EXTERNAL ASSISTANCE

The purpose of this requirement is not to prohibit the use of external resources. Its purpose is to ensure that the origin of significant assistance is honestly acknowledged.

Students are **not required to document ordinary searches or routine consultation of documentation** that do not materially contribute to the submitted work.

RESPONSIBILITY FOR SUBMITTED WORK

Regardless of what resources were used during the development process, each student is fully responsible for understanding the work submitted under their name.

Assignments may be reviewed orally by the instructor.

During such a review, the instructor may ask the student to:

- explain specific lines of code, how an algorithm works;
- explain why a particular implementation was chosen;
- predict what the program will do for a particular input;
- identify or explain an error;
- modify part of the submitted code;
- demonstrate the operation of the submitted program.

RESPONSIBILITY FOR SUBMITTED WORK

The purpose of such a review is to establish that the student understands and is responsible for the submitted work.

A student's inability to explain substantial portions of submitted work, particularly after reasonable questioning and clarification, may be considered as evidence when determining whether the work was genuinely authored by the student. Such evidence will be evaluated together with the other available evidence in accordance with the Montana Tech Student Code of Conduct.

DECLARATION OF RESPONSIBILITY

By submitting an assignment, the student declares that:

1. I am the author of the work I am submitting.
2. I have disclosed any material external assistance used in preparing this work.
3. I understand the code and other work that I am submitting.
4. I take full responsibility for the submitted work.
5. I understand that submitting work that is not my own, or concealing material external assistance, may constitute academic misconduct and may be referred to the appropriate University authority.

The four-line source-file declaration constitutes the student's acknowledgment of these requirements.

UNIVERSITY ACCOMMODATIONS

Students who require academic accommodations should work directly with Montana Tech Disability Services and provide the appropriate documentation to the instructor as soon as possible.

Approved accommodations will be provided in accordance with University policy.

CANVAS

Lecture & Laboratory:

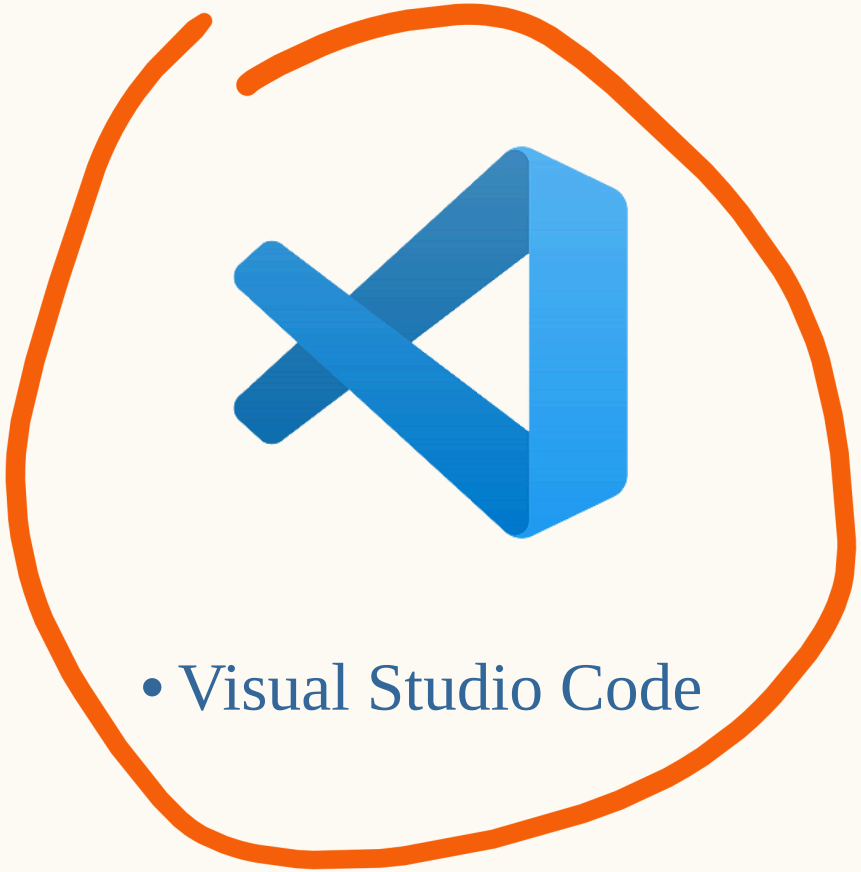
(74801) CSCI 446: ARTIFICIAL INTELLIGENCE

Check if you are registered for the course and have access to it!

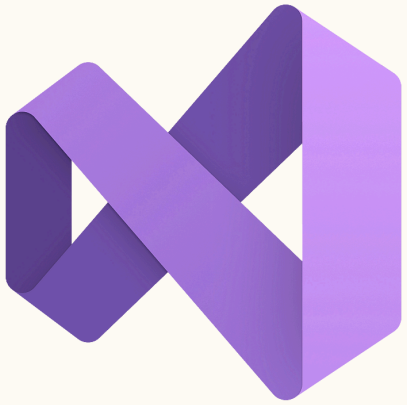
INTEGRATED DEVELOPMENT ENVIRONMENT



- Code::Blocks



- Visual Studio Code



- Visual Studio

THANK

You